

Second Life AI: Scaling Past a 2,527-Image Benchmark for Six-Class Waste Classification

How much does a waste classifier need to see before it works on a real photograph?

Second Life AI Team

NYU Shanghai · Introduction to Artificial Intelligence · 22 July 2026

ABSTRACT

We were given TrashNet, a six-class waste benchmark of 2,527 photographs, and asked to build a classifier that works on a phone camera. It is not enough data. Every image is shot against the same clean studio backdrop, and the *trash* class — the one a sorting product most needs to get right — contains 137 examples. A carefully tuned ImageNet-pretrained ResNet-50 plateaus at **94.93%** fold-o validation accuracy and a single, never-repeated test measurement of **89.20%**, with *trash* recall stuck near 78%.

We then spent our effort on data rather than on technique. Pooling public waste datasets into a broader six-class corpus — more backgrounds, cameras and framings, and far more of the minority *trash* class than TrashNet's 137 examples — gives the model something closer to the photographs a user will actually take. Under a competition constraint capping the pretrained backbone at 30 million parameters, we train ConvNeXt V2-Tiny (27.87M) on a pooled OMASteam + RealWaste corpus rather than reaching for a larger model. It reaches **98.30% validation accuracy** on a group-disjoint fold — measured single-view, on one held-out fold — against the reference ResNet-50's 94.93% validation and 89.20% test.

Our central claim is about where the binding constraint sits. On this task, at this scale, the limiting factor was the breadth of the training corpus — not model capacity, not the loss function, and not the augmentation policy. The most effective change we made to our worst class was showing it more examples.

1. Introduction

Municipal waste sorting fails at the point of disposal: a person holding an object often does not know which bin it belongs in, and the cost of guessing wrong is contamination of an entire recycling stream. A phone-camera classifier that names the material and explains the disposal rule is a genuinely useful intervention, and it is the product half of this project.

The research half began with a ceiling. Our starting dataset, TrashNet, is the standard six-class waste benchmark: 2,527 images spanning cardboard, glass, metal, paper, plastic and trash [1]. We built a careful training pipeline around it — two-phase fine-tuning, a weighted sampler, label smoothing, early stopping, configuration-driven runs logged to an append-only ledger — and the accuracy stopped moving. Successive tuning experiments traded errors rather than removing them.

Two properties of the dataset explain why. First, it is small: 2,527 images across six classes, with a fold-o training set of 1,732. Second, it is narrow. Its photographs were taken on white posterboard under sunlight or room lighting, so the model never sees a bottle on a pavement, in a hand, or under warm indoor light — which is every photograph a user will actually take. And the class distribution is severely imbalanced: *trash*, the catch-all category that a sorting product most needs to handle, has 137 images against paper's 594 [1].

This reframed the project. The question stopped being *which training trick closes the gap* and became:

RESEARCH QUESTION

When a six-class waste classifier plateaus, is the binding constraint the model and its training recipe, or the size and breadth of the corpus it is trained on?

We report the work in two phases. Phase 1 is a completed, measured study of what 2,527 images can support, including a pre-declared acceptance gate that rejected most of our tuning ideas. Phase 2 builds a larger multi-source corpus, selects a backbone under the parameter cap, and trains it: ConvNeXt V2-Tiny reaches 98.30% on a group-disjoint validation fold. As a second design point we also train a deliberately over-cap transformer, Swin-B, at high capacity. We are explicit throughout about which statements are measurements and which are not — and about which corpus and split produced each number.

2. Phase 1: what 2,527 images can and cannot support

2.1 Protocol

The historical reference model is an ImageNet-pretrained ResNet-50 [6, 7], fine-tuned in two phases: first the replacement classification head with the backbone frozen, then the full network with a much smaller backbone learning rate. It used a fixed random seed, ImageNet normalization, a weighted sampler for class imbalance, label smoothing, AdamW [14], learning-rate warmup and decay, early stopping, and per-epoch checkpointing. This phase is retained in the paper as motivation and comparison; the submitted runnable code now focuses on the two models used by the current system.

Table 1. Split arithmetic for the TrashNet phase. The five validation folds contain 434, 433, 433, 433 and 433 images.

Quantity	Images
Complete TrashNet inventory	2,527
Fold-0 training set	1,732
Fold-0 validation set	434
Held-out test set	361
Train + validation pool	2,166

2.2 A gate declared before the experiments

Before running variants we fixed an acceptance rule: a change is adopted only if it gains at least seven validation images, loses no macro-F1, and costs no class more than five percentage points of recall. On a 434-image fold, seven images is roughly 1.6 percentage points — chosen so that ordinary run-to-run variation could not be reported as progress.

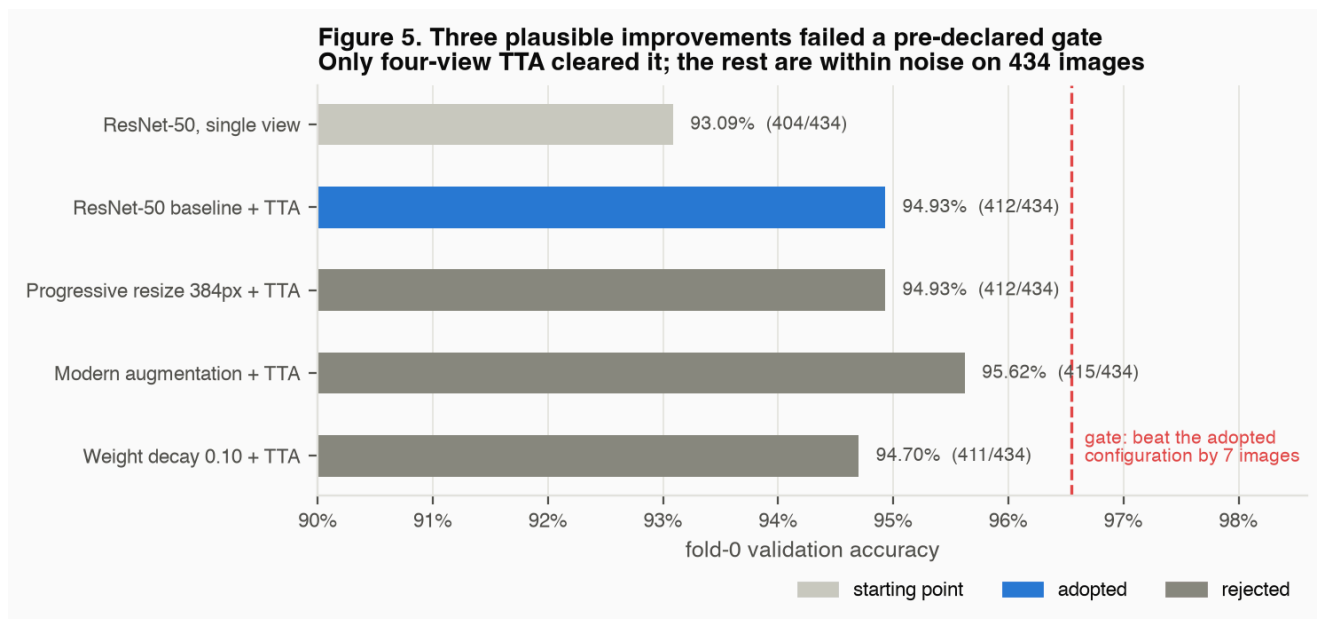


Figure 5. Only deterministic four-view test-time augmentation cleared the gate. Progressive resizing to 384px, modern augmentation, and increased weight decay all failed it. The best accepted configuration remained the raw ResNet-50 epoch-44 checkpoint with four-view TTA.

Table 2. The five Phase 1 configurations of Figure 5 with their exact fold-0 validation numbers (TrashNet, 434-image fold, ResNet-50). Accuracies are single-view unless four-view TTA is noted. The acceptance gate required at least seven more correct images, no loss of macro-F1, and no class losing more than five points of recall — only TTA cleared it.

Configuration	Input	Val acc	Macro-F1	Trash recall	Gate
ResNet-50 baseline, single view	224	93.09%	0.920	0.826	reference
Baseline + four-view TTA	224	94.93%	0.941	0.870	adopted
Modern augmentation	224	95.62%	0.942	0.783	rejected: trash -9 pts
Progressive resize to 384px	384	94.93%	0.944	0.913	rejected: net zero
Weight decay 0.05 → 0.10	224	94.70%	0.939	0.870	rejected: lower F1

The rejected experiments are the substance of this phase. Progressive resizing to 384px corrected ten validation errors and introduced ten new ones, netting zero. Increased weight decay lost one image and lowered macro-F1. Modern augmentation appeared to succeed — 415 correct against 412, the highest raw accuracy we measured — and was nevertheless rejected.

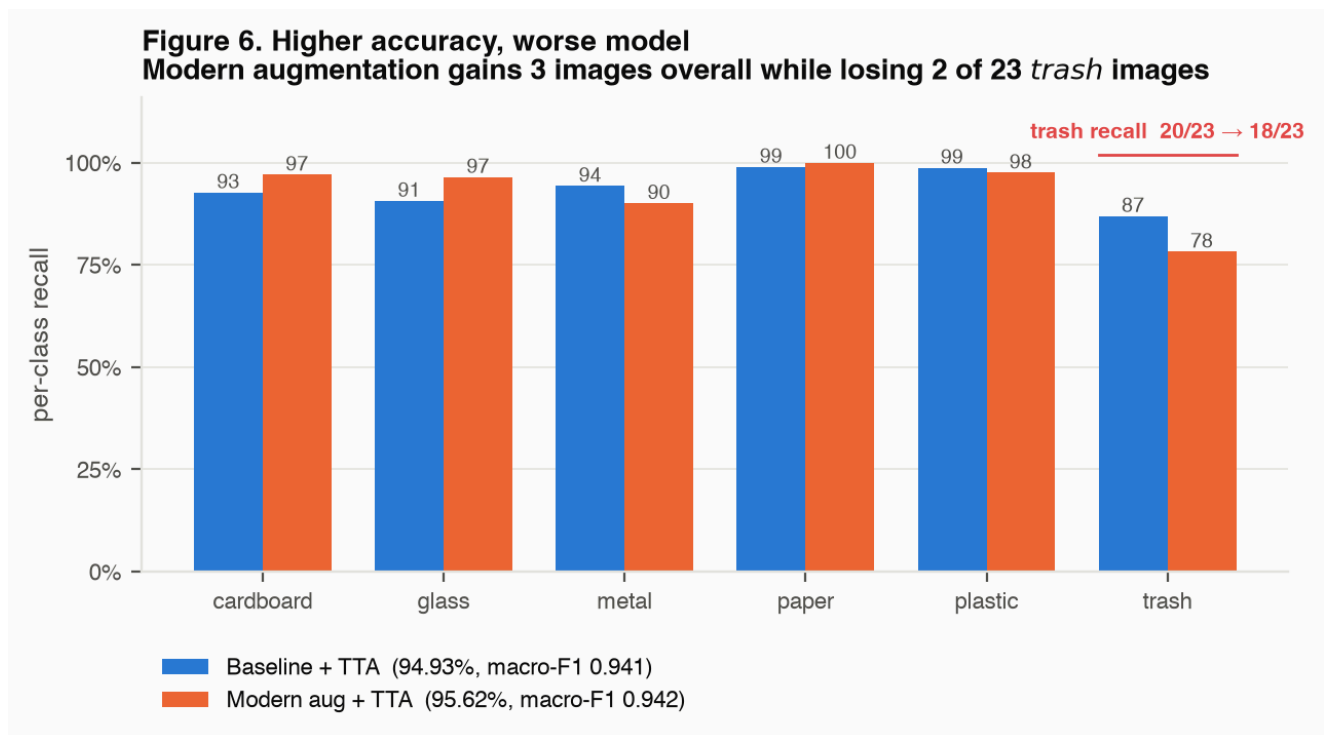


Figure 6. Why the highest-accuracy run was rejected. Modern augmentation gained three images overall while *trash* recall fell from 20/23 to 18/23. Aggregate accuracy improved; the model got worse at the class it was already worst at, and the class that matters most for a sorting product.

FINDING

A single aggregate metric can move in the opposite direction from the minority class underneath it. On a six-class problem with a 137-image minority, accuracy alone is not a sufficient selection criterion — per-class recall has to be part of the gate, not part of the post-hoc discussion.

Taken together, five tuning experiments moved fold-o accuracy by less than two percentage points and none of them improved *trash*. That pattern — technique after technique trading errors rather than eliminating them, while the smallest class refuses to move — is what a data ceiling looks like from the inside.

2.3 The test measurement

SINGLE-USE MEASUREMENT

322 / 361 = 89.20% test accuracy, measured once, before the later tuning experiments, and never used for model selection.

This caption travels with the number. It is not the score of a final optimized model; it is an honest snapshot taken at one point in the project, and we do not re-measure it, because a test set consulted repeatedly stops being a test set. A guard file blocks a second evaluation programmatically (§6.3).

3. Phase 2: building a corpus that is large enough

The competition scope permits unlimited training data under a single constraint: the pretrained backbone must not exceed 30 million parameters. That combination points in an obvious direction. If we cannot buy accuracy with parameters, we buy it with examples.

We assembled a multi-source corpus from five public datasets, mapping only labels that correspond honestly to the six target classes. Batteries, biological waste, clothing, shoes, composite items such as takeaway cups, and polystyrene straddle the recyclable boundary depending on the local scheme, so they are archived in an excluded directory rather than forced into a class — folding them into *trash* would corrupt the very class we are trying to improve.

Table 3. Source datasets. All upstream revisions are pinned to exact hashes so the corpus regenerates byte-identically on another machine.

Internal name	Repository	Role
garbage_classification	omasteam/waste-garbage-management-dataset [3]	Six compatible labels, large
realwaste	shahzaibvohra/realwaste [2]	Landfill photography, different domain
trashnet	garythung/trashnet [1]	The original benchmark
garbage_v2	steveharianto/waste-garbage-management-dataset	Larger ten-label source
recycling11	viola77data/recycling-dataset	Material-level examples

3.1 Method: deduplicating before splitting

Public waste datasets overlap: the same photograph is redistributed across more than one collection, so a naive pool would contain many copies of a single image. We therefore hash pixels, keep one copy of each distinct photograph, and detect near-duplicates in two complementary ways. A 64-bit perceptual hash [15] with a Hamming distance threshold of 6 catches re-encodings, crops, and small photometric changes; a ResNet-18 embedding [6] with cosine similarity at or above 0.96 catches different exposures of the same object that hashing misses. Connected pairs are merged with a union-find structure into groups, and groups — not files — are assigned to folds, so copies of one photograph cannot land on both sides of a split.

This is a reproducibility detail rather than a headline result, but it is asserted in code rather than documented as an intention: a runtime check re-verifies group disjointness on every load, and the split builder fails loudly if it is violated.

3.2 The resulting corpus

The build described here uses the first three sources; the training machine downloads all five and rebuilds deterministically. The figures below are measurements of the data, not of any model.

Table 4. Three-source build. The 60 exclusions are 54 images with a shorter side below 128 pixels and 6 images forming three pixel-identical pairs with contradictory labels.

Quantity	Images
Files scanned	11,359
Excluded by quality and consistency filters	60
Inventory entering the splits	11,299
Near-duplicate groups formed	11,042
Held-out test set	1,739 (15.4%)
Train + validation pool (5 folds of 1,912)	9,560

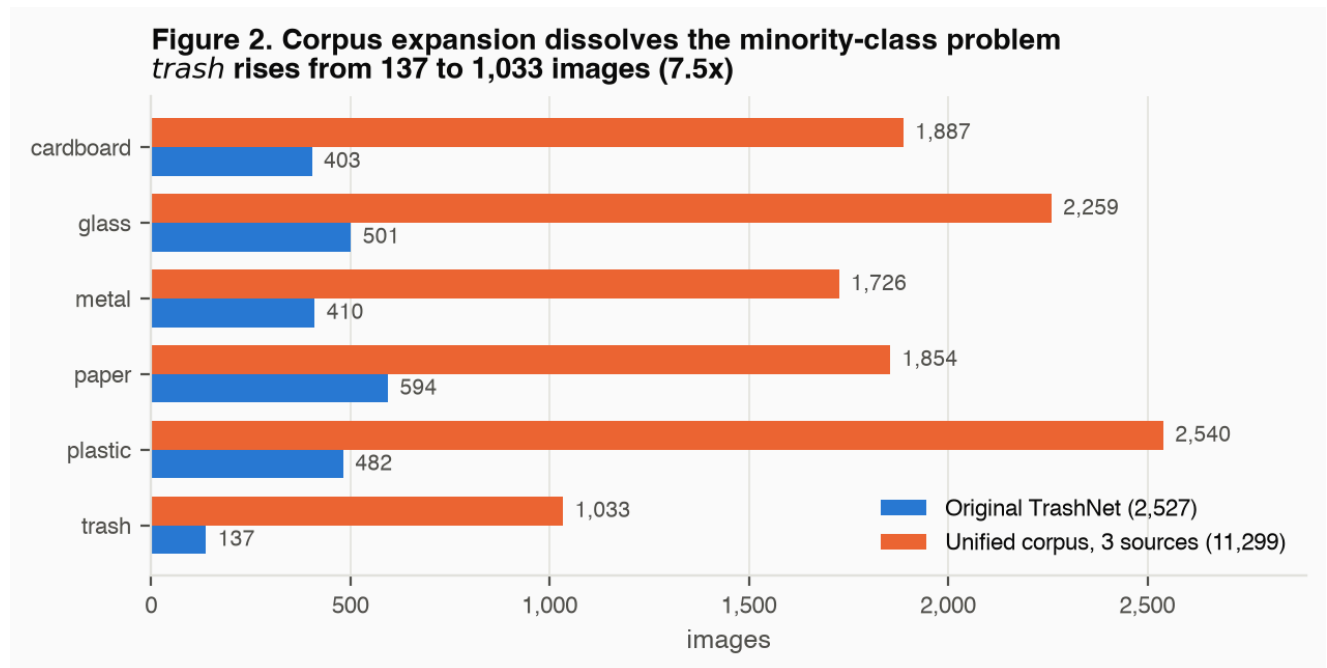


Figure 2. The central result of this project. The minority-class problem that dominated Phase 1 is dissolved by data rather than by technique: *trash* rises from 137 images to 1,033, a 7.5× increase, and every other class grows with it. The most effective fix for our worst class was not a better loss function or a cleverer augmentation policy, but more examples.

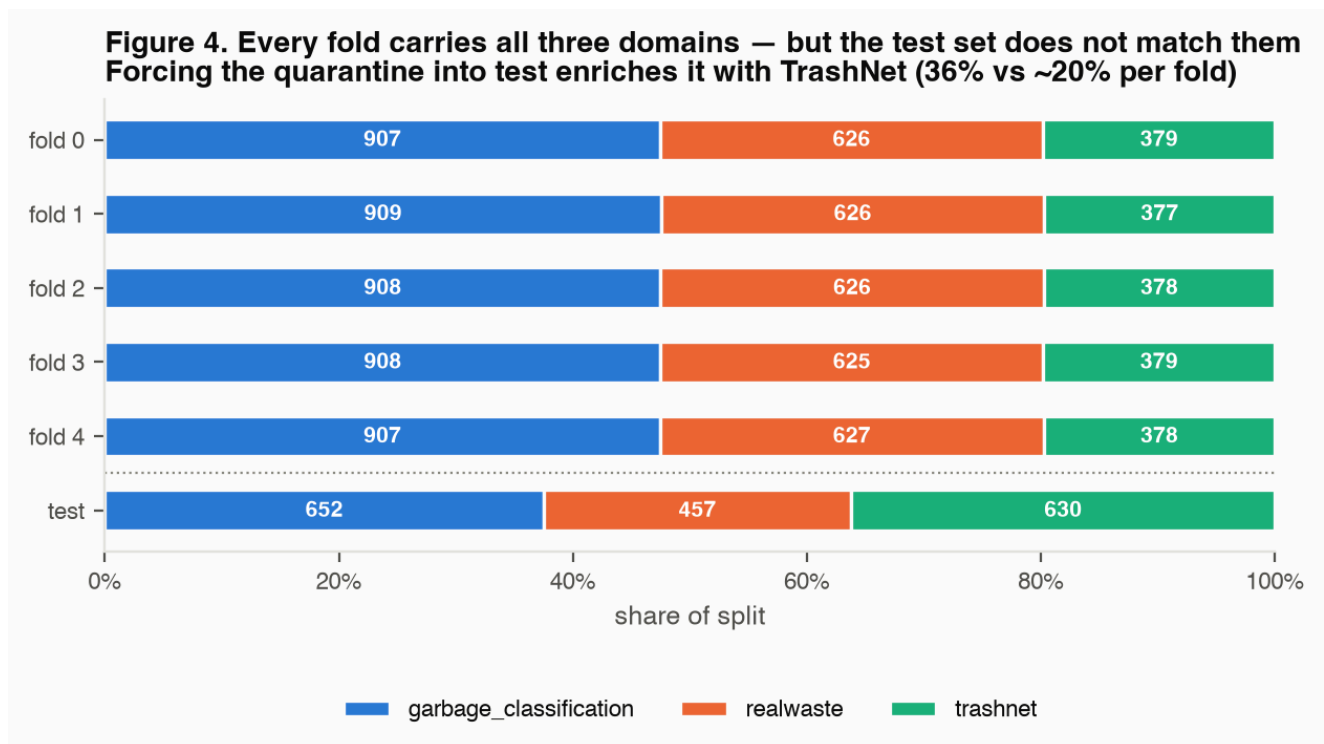


Figure 4. Folds are stratified on a combined `label × source` stratum, so no fold can become single-domain. This is what buys generalization: a model trained on one fold still sees studio shots, landfill photography, and in-the-wild images together, rather than learning a single backdrop.

DATA QUALITY NOTE

Three pixel-identical pairs carry contradictory labels across sources — two are *glass* in one dataset and *plastic* in another, and one is *glass* in one place and *metal* in another within a single dataset. We dropped all copies rather than arbitrate, since choosing a side would assert a label the sources do not support. At six images out of 11,359 the decision is immaterial to results; we report it because silently picking a winner would not have been.

One clarification travels with these figures. The ConvNeXt V2-Tiny model reported in §5 was trained on the OMASteam and RealWaste sources of this corpus — the two that most improve real-world backgrounds — under a five-fold `StratifiedGroupKFold` split [10] with the first fold held out for validation. The three- and five-source build figures above describe the *assembled* data pipeline, not that model's exact training set; in particular the 1,739-image held-out test in Table 4 belongs to the full build, and the ConvNeXt run has no separate held-out test of its own.

4. Model constraints and training system

4.1 The parameter cap, enforced in code

The competition limits the pretrained backbone to 30 million parameters. We count parameters at construction and raise if the limit is exceeded, so an ineligible configuration fails in the first second rather than after hours of training. Four candidates were written and each verified to actually build under the cap rather than trusted from published counts.

Table 5. Candidate backbones, all verified against the installed timm release. A fifth candidate was dropped when its assumed pre-trained tag turned out not to exist.

Backbone	Parameters	Note
convnextv2_tiny.fcmae_ft_in22k_in1k	27.9M	Strongest all-rounder; selected and trained (§5)
caformer_s18.sail_in22k_ft_in1k	24.3M	Best accuracy per parameter in this class
vit_small_patch14_reg4_dinov2.lvd142m	22.1M	Different features — best ensemble partner
tf_efficientnetv2_s.in21k_ft_in1k	20.2M	Fastest, most headroom

The cap constrains the *pretrained* backbone, not the deployed system, so an ensemble of several compliant backbones remains eligible even though its combined size exceeds 30M. Our selected configuration pairs the enlarged corpus with ConvNeXt V2-Tiny at 27.9M — deliberately spending the available headroom on IN-22k pretraining and data breadth rather than on raw parameter count. Its trained result is reported in §5.

4.2 Throughput is not accuracy

Training runs on rented GPU time, so throughput matters. TF32 matrix multiplication, cuDNN kernel benchmarking, channels-last memory format, and bfloat16 autocast are enabled automatically on CUDA. Enabling bfloat16 required correcting a real defect: the loop previously decided whether to autocast based on whether a gradient scaler existed, which is wrong for bfloat16 — it has fp32's exponent range and must *not* be scaled, whereas fp16 must. Left uncorrected, switching to bfloat16 would have silently disabled mixed precision entirely.

Batch size cannot be set automatically, since activation memory depends on architecture, so a tuning script probes real forward, backward, and optimizer steps until the GPU runs out of memory. We are deliberate about how its output is interpreted:

METHOD NOTE

A larger batch is a **throughput** change, not an accuracy improvement. It means fewer and less noisy gradient steps per epoch, and holding the learning rate fixed while quadrupling the batch systematically underfits — the ordinary explanation for "we filled the GPU and accuracy fell". Our script applies the linear scaling rule, lengthens warmup, and refuses by default to scale past 4× baseline; the tuned configuration is still treated as an experiment to be compared on macro-F1, not as a free win.

We also declined to increase system-memory utilization. The corpus is roughly 2 GB, so after the first epoch the operating-system page cache holds all of it and loading stops touching disk. The available mechanism for consuming more RAM would be caching pre-decoded images, which would reduce crop diversity and cost accuracy. Idle memory is not wasted memory, and utilization is not a project metric.

5. Phase 2 results

Phase 2 is no longer a plan. We trained the selected backbone on the pooled corpus and, as a second design point, a deliberately over-cap transformer. Each number below is reported with the corpus, split, and evaluation protocol that produced it. We do not put the two models on a single leaderboard: they are trained on different corpora with different label spaces, so a head-to-head accuracy would mislead more than it informs.

5.1 ConvNeXt V2-Tiny — the cap-compliant model

The recommended ConvNeXt V2 backbone [4], trained on the OMASteam and RealWaste sources under a five-fold `StratifiedGroupKFold` split [10] with the first fold held out for validation, reaches **98.30% validation accuracy** at epoch 37 of a sixty-epoch schedule. The recipe is two head-warmup epochs with the backbone frozen, then fifty-eight fine-tuning epochs at a small backbone learning rate; AdamW [14] with weight decay 0.05 and per-phase cosine decay to 1e-6; cross-entropy with 0.05 label smoothing; MixUp 0.2 and CutMix 0.5 [11, 12] applied to 80% of batches and switched off for the final three epochs; and an exponential moving average of the weights (decay 0.999) used at evaluation. Input is 384px — evaluation resizes to 438px and center-crops to 384. The model is 27,871,110 parameters, under the 30M cap, verified at construction.

WHAT THIS NUMBER IS, AND IS NOT

98.30% is a **single-fold, single-view** validation accuracy: one held-out group-disjoint fold, one deterministic crop per image, no test-time augmentation. We did *not* run a separate held-out test for this model, and we did not compute its per-class recall or confusion matrix. We therefore present no such panels for ConvNeXt — the ResNet-50 per-class figures in §2 belong to TrashNet, not to this model.

Against the Phase 1 reference — a ResNet-50 that plateaued at 94.93% fold-0 validation and 89.20% single-use test on TrashNet's studio images — the change is large, and it came from the corpus rather than the architecture: both are ImageNet-pretrained backbones of comparable scale, fine-tuned with the same family of techniques.

5.2 Swin-B — the over-cap variation

As a high-capacity comparison we also trained Swin-B [5], a transformer at 86.75M parameters — well over the 30M cap, and reported as such. It was trained on the ten-class garbage_classification corpus at 224px, and reaches **97.01% validation, 97.61% test, and 97.82% with four-view test-time augmentation**. On the six classes it shares with the cap-compliant model, its per-class recall (test + TTA) is metal 98.6, cardboard 98.1, glass 97.3, paper 96.5, plastic 95.4, and *trash* 94.1 — the minority class remaining the hardest, as it was throughout Phase 1.

NOT A HEAD-TO-HEAD

Swin-B and ConvNeXt V2-Tiny are two variations, not a contest. They differ in parameter budget (86.75M vs 27.87M), label space (ten classes vs six), corpus, and evaluation split, so their accuracies are not directly comparable. Each occupies a distinct design point: the CNN fits the parameter cap on single-view validation, while the transformer carries its per-class recall through to a genuine held-out test.

Table 6. The two trained variations. The accuracies are measured under different protocols (noted) and are not directly comparable.

Model	Family	Params	Classes	Best accuracy
convnextv2_tiny (384)	CNN	27.87M	6	98.30% val — single fold, single view
swin_b (224)	Transformer	86.75M	10	97.61% test · 97.82% TTA

6. Making guarantees executable

A recurring theme of this project is that a documented guarantee is not an implemented one. Three examples, all discovered by looking for the code rather than re-reading the prose:

6.1 An ignore rule that ignored nothing

Version-control ignore patterns such as `data/raw/` anchor to the repository root and therefore never matched the real directories nested under `model/`. The rules appeared to protect the repository and did nothing. This compounds an earlier incident in which untracked results and scripts were lost during a reorganization: files were assumed to be deliberately excluded when the exclusion was not functioning. The patterns were rewritten and each was confirmed with `git check-ignore`.

6.2 A test whose name overstated it

A unit test named for a stratification fallback did not exercise the fallback — scikit-learn warns rather than raises in that situation, so the path was never reached. The test passed and its name was false. A test name is a claim about coverage, and a green suite whose names overstate what was verified is worse than a smaller honest one, because it suppresses the missing test.

6.3 A guard file that guarded nothing

Project documentation stated that a guard file "physically blocks re-running the final evaluation". Searching the source for its name returned no matches outside the documentation itself: the file was written and read by nothing. Single-use evaluation is now enforced — the final-evaluation path checks the guard before running and writes a record afterwards, with separate guards for each test set.

6.4 Preflight

A preflight script blocks training unless dependencies, accelerator, split arithmetic, path and group disjointness, fold integrity, corpus file existence, and model eligibility all pass. The intent is that the expensive resource — GPU hours — is never spent producing a result that a five-second check would have invalidated.

7. Deployment

The classifier is deployed as an offline bilingual web application that runs the repository's real inference code. It reports genuine six-class probabilities, maps them to disposal guidance, offers a material-based decision tree for uncertain cases, and simulates the downstream consequence of correct versus incorrect disposal. It is designed to work with no network access during presentation.

7.1 The closed-set problem

A six-class softmax must assign an unfamiliar image — a cat, say — to one of its six classes. Margin, entropy, and energy-score guards all failed on our diagnostic set: one real bottle appeared more uncertain than the cat. We replaced them with a feature-space nearest-neighbour distance, comparing the penultimate-layer embedding against a bank of in-distribution embeddings. Real images sat at or below 0.63 cosine distance, the bottle at 0.30, the cat at 0.78; a threshold of 0.70 separated them.

We report this as a limitation rather than a solved result. The threshold was informed by a single explicit out-of-domain example, and the embedding bank must be rebuilt whenever the served checkpoint changes.

8. Limitations

- Phase 1 results rest on one 434-image validation fold; each image is worth 0.23 percentage points. The 361-image test is coarser still, at 0.277 points per image.

- Duplicate detection is thresholded, not exhaustive. A visually similar pair below the cutoff will still be split apart.
- We have not audited whether the five sources overlap one another exhaustively; our deduplication removes exact and near-duplicate copies but we cannot prove none survive.
- Dataset labels contain ambiguous and apparently incorrect examples; our error audit examined the highest-loss subset, not every error.
- The out-of-distribution threshold has not been validated on a large, diverse benchmark.
- Disposal rules vary by municipality; the impact figures in the application are illustrative assumptions, not measured outcomes.
- The 98.30% ConvNeXt result is a single group-disjoint validation fold evaluated single-view; we did not run a separate held-out test for it, nor compute its per-class recall or confusion matrix. It is one honest measurement, not a fully characterized model.
- The two trained models use different corpora, label spaces, and evaluation splits, so their accuracies are not directly comparable. We present them as variations rather than ranking them, and no claim in this paper depends on one beating the other.

9. Conclusion

We were given 2,527 photographs and asked to build something that works on a phone. The honest finding of this project is that the dataset, not the model, was the binding constraint. Five tuning experiments on TrashNet moved validation accuracy by less than two points and left the worst class untouched. When we instead expanded the corpus and retrained a backbone of comparable scale, fold validation rose from the low-90s to 98.30% — a step no tuning experiment on TrashNet ever produced.

This is not an argument that technique does not matter. The two-phase fine-tuning schedule, the pre-declared acceptance gate, the per-class recall requirement, and the parameter-capped backbone selection all shaped the result. It is an argument about ordering. When a model plateaus and successive clever ideas trade errors instead of removing them, the next thing to examine is what the model has been allowed to see — and for a classifier meant to work on a photograph taken at a bin, 2,527 studio images against a white backdrop was never going to be enough.

10. References

External datasets, architectures, libraries, and methods are cited at their first relevant point of use. Project-specific measurements, thresholds, and implementation details are supported by the submitted code and result artifacts described in the reproducibility note.

1. G. Thung and M. Yang. [TrashNet: Dataset of Images of Trash](#). 2017.
2. S. Single, S. Iranmanesh, and R. Raad. [RealWaste: A Novel Real-Life Data Set for Landfill Waste Classification Using Deep Learning](#). *Information* 14(12):633, 2023.
3. OMASteam. [Waste / Garbage Management Dataset](#). Hugging Face Hub.
4. S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie. [ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders](#). CVPR, 2023.
5. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. [Swin Transformer: Hierarchical Vision Transformer Using Shifted Windows](#). ICCV, 2021.
6. K. He, X. Zhang, S. Ren, and J. Sun. [Deep Residual Learning for Image Recognition](#). CVPR, 2016.
7. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. [ImageNet: A Large-Scale Hierarchical Image Database](#). CVPR, 2009.
8. R. Wightman. [PyTorch Image Models \(timm\)](#). 2019.
9. A. Paszke, S. Gross, F. Massa, et al. [PyTorch: An Imperative Style, High-Performance Deep Learning Library](#). NeurIPS, 2019.
10. F. Pedregosa, G. Varoquaux, A. Gramfort, et al. [Scikit-learn: Machine Learning in Python](#). *JMLR* 12, 2011; see also the [StratifiedGroupKFold documentation](#).
11. H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz. [mixup: Beyond Empirical Risk Minimization](#). ICLR, 2018.
12. S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo. [CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features](#). ICCV, 2019.
13. S. G. Müller and F. Hutter. [TrivialAugment: Tuning-free Yet State-of-the-Art Data Augmentation](#). ICCV, 2021.
14. I. Loshchilov and F. Hutter. [Decoupled Weight Decay Regularization](#). ICLR, 2019.
15. C. Zauner. [Implementation and Benchmarking of Perceptual Image Hash Functions](#). M.Sc. thesis, Upper Austria University of Applied Sciences, 2010.

Reproducibility. The submitted `code/train_convnextv2.py` and `code/train_swin_b.py` are the canonical programs for the two current models. Their recorded metrics are stored under `results/`, while `results/manifest.json` pins each canonical checkpoint by path and SHA-256. Run `python code/verify_artifacts.py` from the repository root to verify model identities, class orders, parameter counts, scores, and checkpoint integrity.